

Clasificación de Géneros Musicales mediante XGBoost

Juan Pablo Arteaga Patiño

A01665795

Instituto Tecnológico y de Estudios Superiores de Monterrey

Septiembre 2026

Resumen

En este trabajo se documenta la implementación de un modelo de clasificación multiclase basado en *XGBoost* para predecir el género musical principal de una canción a partir de un conjunto de atributos numéricos y categóricos extraídos de metadatos de pistas musicales. A diferencia de la implementación manual del algoritmo realizada en una primera fase del proyecto, en esta segunda etapa se utilizó un conjunto de datos considerablemente más grande, aprovechando librerías optimizadas de *machine learning* capaces de entrenar de forma eficiente sobre un mayor volumen de información. El conjunto de datos requirió un proceso extenso de extracción, transformación y carga (ETL), que incluyó el aplanamiento de doble encabezado, la conversión de variables de fecha a representación entera, la imputación y codificación *one-hot* de variables categóricas y la eliminación de registros incompletos. Se identificó además un fuerte desbalance entre las clases, lo cual guió decisiones clave de diseño, entre ellas el uso de *f1-macro* como métrica objetivo. Los hiperparámetros se ajustaron mediante una exploración manual e iterativa de configuraciones, de la cual se documentan los cuatro intentos más representativos; se observó que el número de estimadores ($n_estimators$) fue el factor con mayor impacto sobre el desempeño, mientras que la profundidad máxima y la tasa de aprendizaje produjeron variaciones marginales. El modelo final alcanzó una exactitud de 90.72 % y un *f1-macro* de 90.03 % sobre el conjunto de prueba. Se presentan además el análisis de la matriz de confusión normalizada, las curvas ROC y la curva de aprendizaje, con el fin de evaluar tanto el desempeño como la capacidad de generalización del modelo.

1. Introducción

La clasificación automática de géneros musicales es un problema clásico dentro del área de *music information retrieval*, con aplicaciones directas en sistemas de recomendación, organización de bibliotecas musicales y plataformas de streaming. A diferencia de otros problemas de clasificación con clases relativamente balanceadas, la asignación de géneros musicales presenta un reto adicional: las bases de datos reales suelen estar dominadas por unos cuantos géneros populares, mientras que los géneros menos comunes cuentan con muy pocos ejemplos para el aprendizaje del modelo.

El presente trabajo forma parte de la segunda etapa de un proyecto más amplio. En una primera fase se implementó un modelo de regresión logística multinomial de manera manual. Para esta segunda fase, al contar ya con librerías y funciones optimizadas capaces de entrenar modelos de forma más robusta y eficiente, se decidió trabajar con un conjunto de datos considerablemente más grande: un extracto del conjunto de pistas musicales del proyecto FMA [2], cuyo objetivo es predecir el género principal (*track_genre_top*) de una canción a partir de atributos del álbum, del artista y de la propia pista. Este cambio de escala constituye una de las motivaciones centrales del trabajo: el uso de un framework permite explotar volúmenes de información más amplios, a diferencia de modelos manuales.

El objetivo de este documento es describir el proceso de preprocesamiento de datos, la metodología utilizada para el entrenamiento y el ajuste manual de un modelo XGBoost, así como el análisis de los resultados obtenidos, prestando especial atención al efecto que tiene el desbalance de clases sobre las métricas de evaluación y sobre el comportamiento del modelo.

2. Conjunto de Datos y Preprocesamiento

2.1. Fuente de datos

El conjunto de datos utilizado corresponde a *FMA: A Dataset for Music Analysis* [2], obtenido a través de su ficha en el UCI Machine Learning Repository [3] y del repositorio oficial *mdeff/fma* en GitHub [4]. FMA es un conjunto de datos abierto construido a partir del catálogo de **Free Music Archive**, diseñado explícitamente para tareas de análisis musical y que incluye tanto audio como metadatos a nivel de pista, álbum y artista.

En particular, se trabajó con el archivo de metadatos de pistas musicales (*tracks.csv*), el cual contiene información descriptiva y de popularidad a nivel de álbum, artista y pista individual. De la totalidad de columnas disponibles se seleccionó un subconjunto de trece atributos considerados relevantes para la tarea de clasificación, los cuales se resumen en el Cuadro 1.

Tabla 1: Atributos seleccionados del conjunto de datos de pistas musicales.

Origen	Atributo	Tipo
Álbum	Fecha de lanzamiento	Fecha → entero
Álbum	Favoritos	Numérico
Álbum	Número de pistas	Numérico
Álbum	Tipo de álbum	Categorico
Álbum	Reproducciones (<i>listens</i>)	Numérico
Artista	Favoritos	Numérico
Pista	Tasa de bits (<i>bit rate</i>)	Numérico
Pista	Fecha de creación	Fecha → entero
Pista	Duración	Numérico
Pista	Favoritos	Numérico
Pista	Interés (<i>interest</i>)	Numérico
Pista	Número de pista	Numérico
Pista	Género principal (target)	Categorico

Cabe aclarar que estos atributos, no son derivados de la señal de audio (como características espectrales, o sondas sinusoidales), sino de metadatos contextuales y de popularidad. Esta característica del conjunto de atributos condiciona el tipo de patrones que el modelo puede aprender y resulta relevante para interpretar los errores observados en la Sección 4.3.

2.2. Proceso de ETL

A diferencia del conjunto de datos utilizado en la primera fase del proyecto, que se encontraba prácticamente listo para el modelado, el archivo *tracks.csv* requirió un proceso de extracción, transformación y carga (ETL) considerablemente más elaborado. A continuación se describen las etapas aplicadas en el orden en que fueron ejecutadas.

1) Lectura con encabezado jerárquico. El archivo original presenta un encabezado de dos niveles (*multi-index*), en el que el primer nivel identifica el origen de la variable (*album*, *artist*, *track*) y el segundo el atributo concreto (*favorites*, *duration*, etc.). Por ello, la carga debió realizarse indicando explícitamente ambas filas de encabezado y la columna de índice, y la selección de columnas se efectuó mediante tuplas de dos elementos.

2) Selección de atributos. De las decenas de columnas disponibles se retuvieron únicamente los trece atributos del Cuadro 1. Se descartaron columnas redundantes, columnas de texto libre (títulos, biografías, comentarios) y columnas con una proporción de valores nulos tan alta que su imputación habría introducido más ruido que información.

3) Depuración de la variable objetivo. Dado que el objetivo del modelo es predecir el género principal de la pista, se eliminaron todos los registros cuyo valor de *genre_top* fuera nulo. Esta decisión es obligada en un esquema de aprendizaje supervisado: un registro sin etiqueta no aporta información utilizable para el entrenamiento ni puede emplearse para la evaluación. Esta etapa fue, por mucho, la que más registros eliminó, ya que una fracción considerable de las pistas del catálogo carece de género principal asignado.

4) Conversión de variables de fecha. Las columnas de fecha (fecha de lanzamiento del álbum y fecha de creación

de la pista) se encontraban originalmente almacenadas como cadenas de texto. Se convirtieron primero a un tipo de dato de fecha y, posteriormente, a un valor entero de tipo *timestamp*, de forma que pudieran ser utilizadas como variables numéricas ordenadas por el modelo. Esta representación preserva el orden temporal y las distancias relativas entre fechas, lo cual es precisamente la información que un árbol de decisión puede aprovechar e interpretar más fácilmente.

5) Imputación de la variable categórica. La columna de tipo de álbum, al ser categórica y contener valores nulos, se imputó primero con la categoría *unknown*. Se optó por esta estrategia en lugar de eliminar los registros afectados, ya que la ausencia del tipo de álbum puede constituir en sí misma una señal informativa, y descartar dichas filas habría supuesto una pérdida innecesaria de datos.

6) Aplanamiento del encabezado. Una vez realizadas las transformaciones anteriores, los dos niveles del encabezado se concatenaron en un único nombre de columna (por ejemplo, *album_favorites* o *track_duration*). Esto simplifica la manipulación de los datos y asegura que XGBoost sea capaz de trabajar con ellos.

7) One-hot Encoding. La columna de tipo de álbum se codificó mediante variables *dummy* (*one-hot encoding*), dado que el número de categorías distintas era reducido (cinco, incluyendo *unknown*). Esta codificación genera las columnas *album_type_Album*, *album_type_Live Performance*, *album_type_Radio Program*, *album_type_Single Tracks* y *album_type_unknown*.

8) Eliminación final de nulos. Finalmente, una vez completadas todas las transformaciones, se eliminaron los registros restantes con valores nulos en cualquier columna. El dejar esta operación para el final, nos asegura que la información que estamos perdiendo es estrictamente la que no necesitamos.

El resultado es un conjunto de datos limpio y completamente numérico de 49,598 registros y 17 columnas (16 atributos predictores y la variable objetivo), listo para la etapa de modelado.

2.3. Distribución del target y desbalance de clases

Un aspecto central identificado durante el análisis exploratorio es que el conjunto de datos se encuentra fuertemente desbalanceado: la gran mayoría de los registros se concentra en un número reducido de géneros musicales, mientras que otros géneros cuentan con muy pocos ejemplos. La Figura 1 ilustra esta distribución.

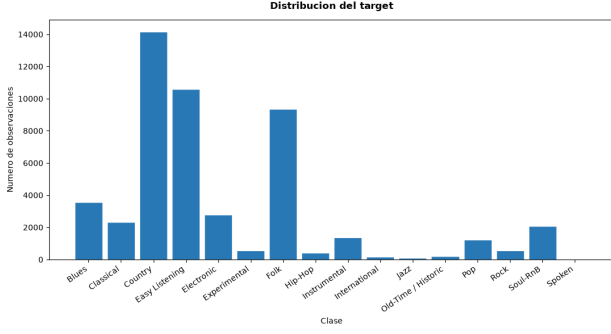


Figura 1: Distribución del target (`track_genre_top`) sobre el conjunto de datos completo.

Géneros como *Rock* (alrededor de 14,000 observaciones), *Experimental* y *Electronic* concentran por sí solos más de dos terceras partes del conjunto de datos, mientras que géneros como *Easy Listening*, *Blues* o *Soul-RnB* cuentan apenas con decenas de ejemplos.

Este hallazgo tuvo un impacto directo sobre varias decisiones de diseño del modelo, particularmente en la elección de la métrica de evaluación utilizada tanto para guiar el ajuste de hiperparámetros como para reportar el desempeño final, según se detalla en la Sección 3.3. También condicionó la forma en que se visualizó la matriz de confusión (Sección 4.3) y la interpretación de los valores de AUC por clase (Sección 4.4).

3. Metodología

3.1. Selección del modelo

Para esta segunda fase del proyecto se optó por utilizar *XGBoost* (*Extreme Gradient Boosting*) [1], un algoritmo de *ensemble* basado en árboles de decisión potenciados por gradiente (*gradient boosting*).

3.2. Pipeline de preprocesamiento

El preprocesamiento numérico se integró dentro de un *pipeline* de *scikit-learn* [5]. Específicamente, se utilizó un *ColumnTransformer* [7] que aplica un escalamiento estándar (*StandardScaler*) a las variables numéricas, mientras que las variables ya codificadas de forma binaria (las variables *dummy* de tipo de álbum).

Cabe señalar que el escalamiento se conservó dentro del *pipeline* con el motivo de dejar el *pipeline* preparado para intercambiar el clasificador por otro que sí sea sensible a la escala; por ejemplo, si en su lugar se quisiera usar *RandomForestClassifier*.

La partición de los datos se realizó con `train_test_split` en una proporción 80/20, con semilla fija (`random_state=67`) para garantizar la reproducibilidad, resultando en 39,678 observaciones de entrenamiento y 9,920 de prueba.

3.3. Métrica de evaluación

Debido al fuerte desbalance de clases descrito en la Sección 2.3, la elección de la métrica utilizada tanto para guiar el ajuste de hiperparámetros como para evaluar el desempeño final del modelo resultó crítica. Se decidió utilizar *f1-macro* en lugar de *accuracy*, *f1-micro* o *f1-weighted*.

Para una clase k dada, la precisión y el *recall* se definen como

$$P_k = \frac{TP_k}{TP_k + FP_k}, \quad R_k = \frac{TP_k}{TP_k + FN_k}, \quad (1)$$

y el *f1-score* de esa clase como su media armónica:

$$F1_k = 2 \frac{P_k \cdot R_k}{P_k + R_k}. \quad (2)$$

El promedio macro se obtiene entonces como la media aritmética simple sobre las K clases:

$$F1_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K F1_k, \quad (3)$$

mientras que el promedio ponderado introduce el soporte n_k de cada clase:

$$F1_{\text{weighted}} = \frac{1}{N} \sum_{k=1}^K n_k F1_k. \quad (4)$$

La diferencia se hace más importante en un contexto desbalanceado [12, 8]. El *f1-macro* calcula la métrica de manera independiente para cada clase y después promedia los resultados sin ponderar por el número de observaciones, de modo que todos los géneros musicales (independientemente de cuántos ejemplos tengan) contribuyen de manera equitativa a la métrica final. Un modelo que no aprenda por completo de *Easy Listening* y de *Blues* sufriría una penalización de la igual que si ignorase a *Rock*.

Si se hubiese utilizado *f1-weighted* habría otorgado mayor peso a los géneros más representados (como *Rock* o *Experimental*), lo que se entiende como un modelo que busca acertar los géneros con mayores ejemplares, sin preocuparse en aprender de los minoritarios. El caso extremo es la *accuracy*: en este conjunto de datos, un clasificador que asignara siempre la clase *Rock* obtendría cerca de un 28 % de aciertos sin haber aprendido absolutamente nada, y su *f1-macro* sería de apenas 0.027. Al utilizar *f1-macro* como métrica objetivo se buscó forzar al proceso de optimización a favorecer configuraciones capaces de generalizar bien sobre todas las clases del problema, y no únicamente sobre las más frecuentes [13].

Como métricas complementarias se reportan también la *accuracy*, la exactitud balanceada (promedio macro del *recall*) y el *f1-weighted*, ya que la comparación entre ellas resulta informativa: una brecha grande entre *f1-weighted* y *f1-macro* sería síntoma de un modelo sesgado hacia las clases mayoritarias.

3.4. Ajuste manual de hiperparámetros

Se optó por una exploración manual e iterativa, en la que cada configuración se evaluó sobre el conjunto de prueba y

los resultados guiaron la elección de la siguiente configuración a probar.

Los hiperparámetros considerados fueron:

- `n_estimators`: número de árboles del *ensemble*, es decir, de iteraciones de *boosting*.
- `max_depth`: profundidad máxima de cada árbol, que controla la complejidad de las interacciones que cada aprendiz débil puede capturar.
- `learning_rate`: factor de contracción aplicado a la contribución de cada árbol.
- `subsample`: fracción de observaciones muestreada aleatoriamente para construir cada árbol, que introduce un componente estocástico con efecto regularizador.

De la exploración realizada se documentan los cuatro intentos más representativos, resumidos en el Cuadro 2. Las configuraciones se eligieron de modo que fueran comparables entre sí: los intentos 1, 2 y 4 varían únicamente el número de estimadores manteniendo el resto de parámetros constantes, mientras que los intentos 1 y 3 varían únicamente la profundidad máxima. La tasa de aprendizaje se fijó en 0.1 y el submuestreo en 0.8 en los cuatro casos documentados, después de que pruebas exploratorias previas mostraran que variaciones en dichos parámetros no producían diferencias marcadas en el desempeño una vez fijado el número de árboles.

Incrementar el número de estimadores de 200 a 300 elevó el *f1-macro* de 0.7394 a 0.8280 (+0,089), y pasar de 300 a 500 lo elevó nuevamente hasta 0.9003 (+0,072). En contraste, reducir la profundidad máxima de 5 a 3 con 200 estimadores (intento 3 frente a intento 1) modificó el *f1-macro* en apenas -0,022. Es decir, el efecto del número de árboles resultó aproximadamente cuatro veces mayor.

Es importante notar que el aumento de *n_estimators* beneficia de manera especialmente notoria a las clases minoritarias. Comparando los reportes por clase de los intentos 1 y 4, el *recall* de *Folk* pasó de 0.37 a 0.79 y el de *Pop* de 0.38 a 0.79, mientras que el de *Rock* solo subió de 0.83 a 0.93. Esto sugiere que las primeras iteraciones de *boosting* se dedican principalmente a modelar las clases dominantes, y que las clases con menor soporte requieren un número considerablemente mayor de iteraciones para que el residual asociado a ellas comience a ser corregido.

Con base en estos resultados se seleccionó la configuración del intento 4 como modelo final.

4. Resultados

4.1. Métricas generales

El modelo final, evaluado sobre el conjunto de prueba (20 % del total de los datos, equivalente a 9,920 observaciones), obtuvo los resultados generales resumidos en el Cuadro 3.

Tabla 3: Métricas generales del modelo XGBoost final sobre el conjunto de prueba.

Métrica	Valor
Exactitud (<i>Accuracy</i>)	0.9072
Exactitud Balanceada	0.8643
F1 Macro	0.9003
F1 Weighted	0.9069
AUC micro (<i>one-vs-rest</i>)	1.00
AUC macro (<i>one-vs-rest</i>)	0.98

Es relevante notar que el valor de *accuracy* (90.72 %) y el de *f1-macro* (90.03 %) resultaron muy cercanos entre sí. Como se argumentó en la Sección 3.3, en un conjunto de datos tan desbalanceado como este, un modelo que simplemente favoreciera a las clases mayoritarias para inflar su exactitud mostraría un *f1-macro* inferior. La cercanía entre ambos valores, así como entre el *f1-macro* y el *f1-weighted* (0.9069), lo estoy interpretando como indicio favorable de que el modelo mantiene un desempeño razonablemente consistente entre clases y no solo aprende a clasificar las mayoritarias, que en principio era el objetivo de comparar las distintas métricas que seleccioné pensadas para este conjunto de datos.

La exactitud balanceada (0.8643), que equivale al promedio macro del *recall*, es la métrica más baja del conjunto. Esta diferencia frente al *f1-macro* puede llegar a indicar que el modelo es más preciso que sensible, es decir, cuando predice una clase minoritaria suele acertar, pero no logra recuperar todos los casos verdaderos de dicha clase.

4.2. Resultados por clase

El Cuadro 4 presenta el reporte de clasificación completo, con las métricas de precisión, *recall* y *f1-score* para cada uno de los 16 géneros musicales considerados, junto con el número de observaciones de prueba (*support*) correspondiente a cada clase.

Tabla 2: Comparación de las cuatro configuraciones de hiperparámetros documentadas. Todas las métricas fueron evaluadas sobre el mismo conjunto de prueba (9,920 observaciones). La configuración seleccionada aparece resaltada.

Intento	Hiperparámetros				Métricas sobre el conjunto de prueba			
	n_estimators	max_depth	learning_rate	subsample	Accuracy	Bal. Accuracy	F1 Macro	F1 Weighted
1	200	5	0.1	0.8	0.7151	0.6712	0.7394	0.7096
2	300	5	0.1	0.8	0.8065	0.7709	0.8280	0.8050
3	200	3	0.1	0.8	0.6931	0.6521	0.7170	0.6847
4	500	5	0.1	0.8	0.9072	0.8643	0.9003	0.9069

Tabla 4: Reporte de clasificación por género sobre el conjunto de prueba (modelo final, intento 4).

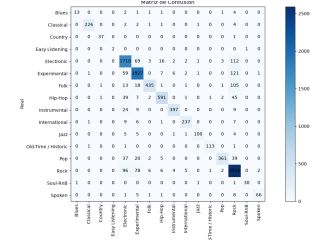
Género	Precisión	Recall	F1	Soporte
Blues	1.00	0.52	0.69	23
Classical	0.98	0.97	0.97	237
Country	0.93	0.97	0.95	38
Easy Listening	1.00	0.67	0.80	3
Electronic	0.87	0.91	0.89	1,926
Experimental	0.91	0.92	0.92	2,125
Folk	0.95	0.79	0.86	575
Hip-Hop	0.96	0.90	0.93	689
Instrumental	0.97	0.92	0.94	439
International	0.98	0.95	0.96	261
Jazz	0.97	0.89	0.93	116
Old-Time / Historic	0.99	0.97	0.98	116
Pop	0.96	0.79	0.87	464
Rock	0.87	0.93	0.90	2,793
Soul-RnB	0.97	0.91	0.94	33
Spoken	0.94	0.83	0.88	82
Promedio macro	0.95	0.86	0.90	9,920
Promedio ponderado	0.91	0.91	0.91	9,920

Las clases con menor *recall* (*Blues* con 0.52, *Easy Listening* con 0.67, *Folk* y *Pop* con 0.79) corresponden mayoritariamente a géneros con pocos ejemplares o que son muy similares a clases dominantes, lo cual es consistente con la dificultad esperada de aprender patrones robustos a partir de pocos ejemplares.

Conviene señalar que las métricas de las clases más pequeñas son muy frágiles. Con únicamente 3 observaciones de prueba, el *recall* de *Easy Listening* solo puede tomar los valores 0, 0.33, 0.67 o 1.00; un solo acierto adicional o un solo error modifican la métrica en 33 puntos porcentuales. Lo mismo ocurre con *Blues* (23 observaciones) y *Soul-RnB* (33). Por tanto, no hay mucho de donde sacar mayores conclusiones sobre estas clases.

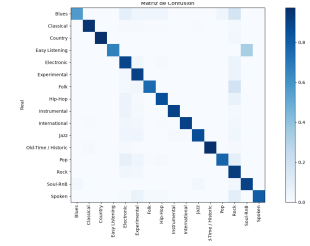
4.3. Matriz de confusión

La Figura 2 muestra la matriz de confusión sin normalizar sobre el conjunto de prueba, mientras que la Figura 3 presenta la versión normalizada por fila; es decir, cada fila representa la proporción de observaciones reales de una clase que fueron asignadas a cada categoría predicha.

**Figura 2:** Matriz de confusión (valores absolutos) del modelo XGBoost sobre el conjunto de prueba.

Dado que hay pocos géneros que concentran la mayoría de las observaciones, la matriz de confusión sin normalizar tiende a opacar el comportamiento del modelo sobre las clases minoritarias: las celdas correspondientes a los géneros mayoritarios dominan visualmente la escala de color (la celda de *Rock* acumula 2,599 aciertos, mientras que toda la fila de *Easy Listening* suma 3 observaciones) lo que hace prácticamente imposible apreciar patrones de error en las clases con menos ejemplos, que aparecen como celdas casi blancas indistinguibles entre sí. Por esta razón opté por analizar principalmente la versión normalizada, en la que cada fila se divide entre su propio total y, en consecuencia, la intensidad del color codifica proporciones comparables entre clases independientemente de su tamaño.

Del análisis de la matriz normalizada se desprenden dos observaciones principales.

**Figura 3:** Matriz de confusión normalizada por fila del modelo XGBoost sobre el conjunto de prueba. La normalización permite comparar el comportamiento del modelo entre clases con soportes muy distintos.

Al ser **Rock** la clase con mayor número de observaciones, es la que acumula el mayor número de aciertos y, en términos absolutos, el mayor número de errores propios. Sin embargo, mirando su columna, podemos notar que: *Rock* es el destino de errores procedentes de casi todas las demás clases. En concreto, 89 de las 575 observaciones de *Folk* (15.5 %), 91 de *Experimental*, 104 de *Electronic* y 37

de *Pop* fueron clasificadas erróneamente como *Rock*. De la misma manera, los errores del propio *Rock* se dirigen mayoritariamente hacia *Electronic* (86) y *Experimental* (74). Es decir, existe una zona de confusión mutua entre *Rock*, *Folk*, *Experimental* y *Electronic*, con un sesgo claro hacia *Rock*. Una lectura que puedo dar, es que hay una similitud en las características de metadatos entre estos géneros, que en el catálogo de FMA comparten perfiles de duración, tipo de álbum y popularidad.

Obviamente, esto es una interpretación más no una respuesta verídica; lo dejo en claro porque pude también identificar otro fenómeno que explique de manera distinta estos segos:

En la matriz normalizada destaca una celda fuera de la diagonal, correspondiente al par *Easy Listening–Soul-RnB*, una de las tres observaciones de *Easy Listening* (33 %) fue clasificada como *Soul-RnB*. Esto podría deberse, como ya lo mencioné antes, a una similitud real en las características de ambos género, pero también puede deberse al hecho de que *Easy Listening* es, por un margen considerable, el género con menor número de observaciones del conjunto de datos, lo cual limita severamente la capacidad del modelo para aprender un patrón suficientemente robusto y distintivo para dicha clase.

4.4. Curva ROC

El área bajo la curva (AUC) promedio micro fue de 1.00, mientras que el promedio macro fue de 0.98. La cercanía entre ambos valores es, nuevamente, un indicio de que el modelo mantiene un buen poder discriminativo incluso al considerar cada clase de manera individual como lo establecimos en la métrica. La mayoría de las clases individuales presentan un AUC igual o superior a 0.98, con la excepción de *Easy Listening* (AUC = 0,74).

4.5. Curva de aprendizaje

Finalmente, se generó la curva de aprendizaje del modelo, evaluando el *f1-macro* tanto en el conjunto de entrenamiento como en el de validación conforme se incrementa el número de ejemplos utilizados para entrenar (Figura 5). El procedimiento corresponde al descrito en la documentación de `learning_curve` de *scikit-learn* [9], utilizando validación cruzada de tres particiones y cuatro tamaños de entrenamiento distribuidos uniformemente entre el 30 % y el 100 % del conjunto disponible.

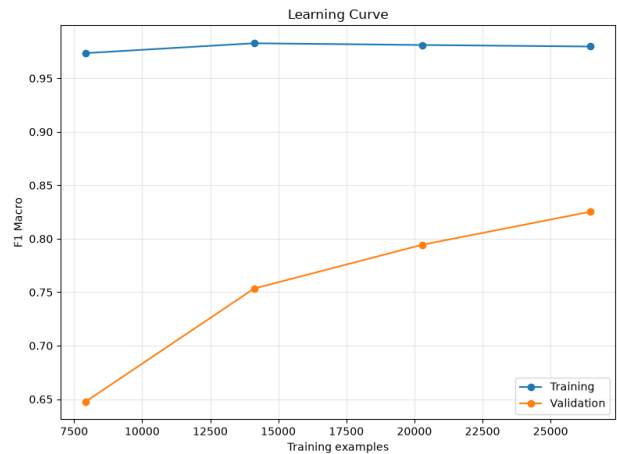


Figura 5: Curva de aprendizaje del modelo XGBoost (*f1-macro*).

Tabla 5: Valores aproximados de la curva de aprendizaje y evolución de la brecha entre entrenamiento y validación.

Ejemplos	F1 entren.	F1 valid.	Brecha
7,936	0.978	0.663	0.315
14,109	0.995	0.762	0.233
20,282	0.990	0.807	0.183
26,452	0.988	0.839	0.149

Se observa que la métrica de entrenamiento se mantiene consistentemente alta y prácticamente constante (alrededor de 0.98–0.99) a lo largo de todos los tamaños de conjunto evaluados, mientras que la métrica de validación, aunque muestra una tendencia claramente creciente conforme aumenta el número de ejemplos de entrenamiento (pasando de aproximadamente 0.66 con cerca de 8,000 ejemplos a 0.84 con el conjunto completo), permanece consistentemente por debajo de la métrica de entrenamiento.

El modelo se beneficiaría de contar con más datos de entrenamiento, que dada la pendiente todavía positiva de la curva de validación, muy probablemente continuaría cerrando la brecha entre ambas curvas; el conjunto FMA completo, que contiene más de 100,000 pistas, pudiera solucionar este matiz; el problema se encuentra en que muchos de los datos no son elegibles para entrenamiento.

Aquí quiero señalar un punto muy importante. La curva de aprendizaje emplea validación cruzada de tres particiones de manera **aleatoria**, de modo que los modelos se entrenan y se evalúan sobre particiones en las que las clases minoritarias pueden estar sub-representadas o incluso ausentes. Al tratarse de una métrica macro, la ausencia o el desempeño errático de una sola clase minoritaria penaliza el promedio de forma desproporcionada. La cifra de 0.84 debe leerse, por tanto, como una estimación conservadora obtenida bajo condiciones menos favorables, y no como una contradicción del resultado sobre el conjunto de prueba. De nuevo, sería más conveniente obtener más metadatos de las clases minoritarias para evitar este tipo de contradicciones al momento de evaluar nuestro modelo, puesto que la métrica elegida, en este caso, desfavorece muchísimo el resultado de nuestro desempeño en la curva de aprendizaje.

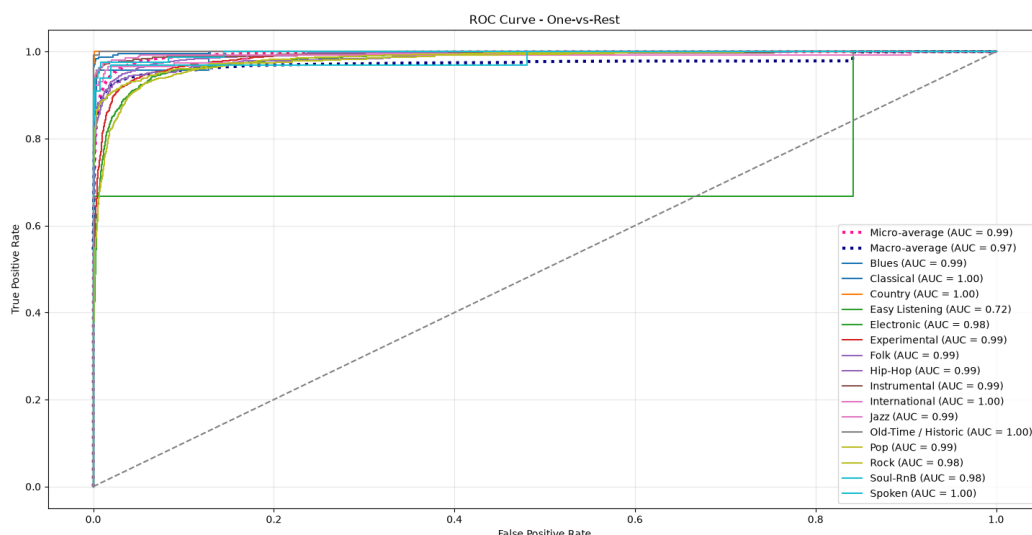


Figura 4: Curvas ROC *one-vs-rest* por género, con promedios micro (AUC = 1,00) y macro (AUC = 0,98).

5. Conclusiones

Los resultados obtenidos permiten concluir que el modelo XGBoost, combinado con un ajuste de hiperparámetros orientado explícitamente a maximizar el desempeño equilibrado entre clases (*f1-macro*), logra un desempeño sólido sobre un conjunto de datos con un desbalance considerable. La cercanía entre las métricas de *accuracy* (0.9072), *f1-macro* (0.9003) y *f1-weighted* (0.9069) indica que el modelo no está simplemente memorizando el comportamiento de las clases mayoritarias, sino que logra generalizar razonablemente bien incluso hacia géneros con pocas observaciones.

Los patrones de confusión identificados — particularmente entre *Rock*, *Folk*, *Experimental* y *Electronic*, así como entre *Easy Listening* y *Soul-RnB*— sugieren que existe una similitud real en las características de metadatos entre dichos grupos de géneros. Aquí conviene recordar la naturaleza de los atributos empleados: el modelo no tiene acceso a ninguna propiedad acústica de la señal, sino únicamente a metadatos de catálogo y popularidad. Al mismo tiempo, explica de forma natural por qué géneros estilísticamente vecinos, publicados por sellos y comunidades similares dentro de la misma plataforma, resultan difíciles de separar.

Con el fin de no sobre-complicar el modelo, muchos de los datos referentes al nombre u origen de las canciones fueron desechados, puesto que no aportan nada al modelo por la naturaleza de estos. Pero sería útil encontrar una manera de obtener datos acústicos (llamase *tempo*, *características espectrales*, *registros de rango*, *ondas sinusoidales*, etc) de cada canción a partir de alguna búsqueda exhaustiva tomando como referencia títulos, fechas de publicación y artistas, con el objetivo de utilizar estos nuevos parámetros para distinguir y discriminar más eficientemente los géneros.

En conjunto, los resultados confirman la viabilidad de utilizar XGBoost, acompañado de un ajuste cuidadoso de hiperparámetros y de una métrica de evaluación adecuada al contexto de desbalance de clases, como una herramienta robusta para la clasificación de géneros musicales a partir

de metadatos.

Referencias

- [1] Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.
- [2] Defferrard, M., Benzi, K., Vandergheynst, P., & Bresson, X. (2017). *FMA: A Dataset for Music Analysis*. 18th International Society for Music Information Retrieval Conference (ISMIR).
- [3] Defferrard, M., Benzi, K., Vandergheynst, P., & Bresson, X. (2017). *FMA: A Dataset for Music Analysis* [Conjunto de datos]. UCI Machine Learning Repository. DOI: 10.24432/C5HW28. <https://archive.ics.uci.edu/dataset/386/fma+a+dataset+for+music+analysis>
- [4] Defferrard, M., y colaboradores. *FMA: A Dataset for Music Analysis* [Repositorio de código y recursos]. GitHub. <https://github.com/mdeff/fma>
- [5] Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
- [6] XGBoost Developers. *XGBoost Documentation* (incluye *XGBoost Parameters* y *Notes on Parameter Tuning*). <https://xgboost.readthedocs.io/>
- [7] Scikit-learn. *ColumnTransformer*. <https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html>
- [8] Scikit-learn. *f1_score*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html
- [9] Scikit-learn. *learning_curve*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.learning_curve.html

- [10] Scikit-learn. *Multiclass Receiver Operating Characteristic (ROC)*. https://scikit-learn.org/stable/auto_examples/model_selection/plot_roc.html
- [11] Fawcett, T. (2006). *An Introduction to ROC Analysis*. Pattern Recognition Letters, 27(8), 861–874.
- [12] Grandini, M., Bagli, E., & Visani, G. (2020). *Metrics for Multi-Class Classification: an Overview*. arXiv preprint arXiv:2008.05756.
- [13] Brownlee, J. *How to Configure XGBoost for Imbalanced Classification*. Machine Learning Mastery. <https://machinelearningmastery.com/xgboost-for-imbalanced-classification/>
- [14] Brownlee, J. *How to Tune the Number and Size of Decision Trees with XGBoost in Python*. Machine Learning Mastery. <https://machinelearningmastery.com/tune-number-size-decision-trees-xgboost-python/>
- [15] Brownlee, J. *Tune Learning Rate for Gradient Boosting with XGBoost in Python*. Machine Learning Mastery. <https://machinelearningmastery.com/tune-learning-rate-for-gradient-boosting-with-xgboost-in-python/>
- [16] Brownlee, J. *How to use Learning Curves to Diagnose Machine Learning Model Performance*. Machine Learning Mastery. <https://machinelearningmastery.com/learning-curves-for-diagnosing-machine-learning-model-performance/>
- [17] Jain, A. (2016). *Complete Guide to Parameter Tuning in XGBoost with codes in Python*. Analytics Vidhya. <https://www.analyticsvidhya.com/blog/2016/03/complete-guide-parameter-tuning-xgboost-with-codes-python/>